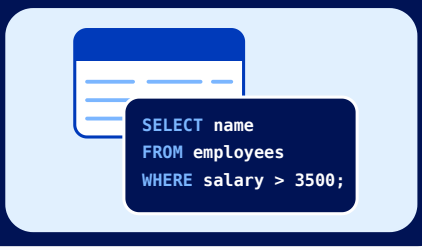
CP03 PROGRAMMING & SOFTWARE DEVELOPMENT

SQL CHEAT SHEET

Essential SQL Commands for Beginners



APA ITU SQL?

SQL (Structured Query Language) ialah bahasa untuk berkomunikasi dengan relational database system. Digunakan untuk:

- Retrieve data
- Filter records
- Sort results
- Combine related tables
- Insert new records
- Modify existing records
- Delete records

STRUKTUR QUERY ASAS

```
SELECT column_name
FROM table_name
WHERE condition;
```

SELECT	pilih column yang nak diambil.
FROM	tentukan table sumber.
WHERE	tapis rekod ikut condition.

Nota. Sesetengah syntax dan behaviour berbeza antara database system (MySQL, PostgreSQL, SQL Server, SQLite). Contoh di sini guna syntax biasa untuk pembelajaran asas dan diuji pada SQLite.

company_db Satu database contoh yang sama digunakan sepanjang cheatsheet ni.

employees

employee_id	name	department_id	salary	city
101	Ali	1	3500	Kuala Lumpur
102	Sara	2	4200	Cyberjaya
103	Daniel	1	3800	Shah Alam
104	Aina	3	4500	Putrajaya
105	Amir	2	3200	Cyberjaya

departments

department_id	department_name
1	IT
2	Marketing
3	Finance

01 SELECT

Ambil data daripada database.

SYNTAX

```
SELECT column_name
FROM table_name;
```

CONTOH

```
SELECT name, salary
FROM employees;
```

SEMUA COLUMN

```
SELECT *
FROM employees;
```

HASIL

name	salary
Ali	3500
Sara	4200
Daniel	3800
Aina	4500
Amir	3200

PENTING `SELECT *` ambil semua column daripada table. Senang untuk belajar dan explore, tapi dalam production query biasanya lebih baik pilih column yang diperlukan sahaja.

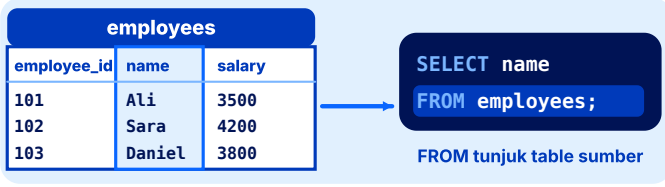
02 FROM

Tentukan table atau sumber data untuk rekod diambil.

CONTOH

```
SELECT name
FROM employees;
```

`employees` ialah table sumber. Semua column dalam SELECT mesti datang daripada sumber ni.



03 WHERE

Tapis rekod berdasarkan condition tertentu.

```
SYNTAX
SELECT column_name
FROM table_name
WHERE condition;
```

```
CONTOH
SELECT *
FROM employees
WHERE department_id = 1;
```

OPERATOR	MAKSUD
=	Sama dengan
<>	Tak sama
>	Lebih besar
<	Lebih kecil
>=	Besar atau sama
<=	Kecil atau sama

OPERATOR	MAKSUD
AND	Kedua-dua benar
OR	Salah satu benar
LIKE	Pattern matching
IN	Nilai dalam senarai
BETWEEN	Dalam julat
IS NULL	Semak NULL

HASIL Ali, Daniel (2 baris)

```
SELECT *
FROM employees
WHERE salary > 3500;
```

HASIL Sara, Daniel, Aina (3 baris)

```
SELECT *
FROM employees
WHERE city = 'Cyberjaya';
```

HASIL Sara, Amir (2 baris)

```
SELECT *
FROM employees
WHERE department_id IN (1, 2);
```

HASIL Ali, Sara, Daniel, Amir (4 baris)

```
SELECT *
FROM employees
WHERE name LIKE 'A%';
```

HASIL Ali, Aina, Amir (3 baris)

```
SELECT *
FROM employees
WHERE department_id = 2 AND salary > 3500;
```

HASIL Sara (1 baris)

```
SELECT *
FROM employees
WHERE salary BETWEEN 3500 AND 4200;
```

HASIL Ali, Sara, Daniel (3 baris)

`'A%'` padan nama yang bermula dengan huruf A. LIKE boleh jadi case-sensitive atau tak, bergantung pada database system.

PENTING Guna `IS NULL`, bukan `= NULL`, bila semak nilai NULL.

04 ORDER BY

Susun hasil query.

```
SYNTAX
SELECT column_name
FROM table_name
ORDER BY column_name ASC;
```

```
ASCENDING
SELECT *
FROM employees
ORDER BY salary ASC;
```

```
DESCENDING
SELECT *
FROM employees
ORDER BY salary DESC;
```

ASC susunan menaik (kecil ke besar).

DESC susunan menurun (besar ke kecil).

ASC biasanya ialah arah susunan default bila tak dinyatakan.

Asal (name, salary)

name	salary
Ali	3500
Sara	4200
Daniel	3800
Aina	4500
Amir	3200

DESC

ORDER BY salary DESC

name	salary
Aina	4500
Sara	4200
Daniel	3800
Ali	3500
Amir	3200

05 GROUP BY

Kumpul rows yang kongsi nilai sama. Biasanya digunakan dengan aggregate function.

SYNTAX

```
SELECT column_name, aggregate_function()
FROM table_name
GROUP BY column_name;
```

CONTOH

```
SELECT department_id, COUNT(*) AS total_employees
FROM employees
GROUP BY department_id;
```

Query ni kira bilangan pekerja dalam setiap department.

FUNCTION	TUJUAN
`COUNT()`	Kira rows atau nilai bukan NULL, bergantung pada expression
`SUM()`	Kira jumlah
`AVG()`	Kira purata
`MIN()`	Cari nilai minimum
`MAX()`	Cari nilai maksimum

HASIL CONTOH

department_id	total_employees
1	2
2	2
3	1

CONTOH LAIN

```
SELECT department_id, AVG(salary) AS average_salary
FROM employees
GROUP BY department_id;
```

HASIL CONTOH LAIN

department_id	average_salary
1	3650
2	3700
3	4500

PENTING

- Column dalam SELECT yang bukan aggregate biasanya perlu ada dalam GROUP BY, tertakluk pada peraturan SQL dialect dan functional dependency.
- GROUP BY bukan sekadar susun data. Ia kumpul rows untuk aggregation.
- Guna ORDER BY kalau nak susun group yang terhasil.

BILA COMMAND DIGABUNG: WHERE, GROUP BY DAN ORDER BY

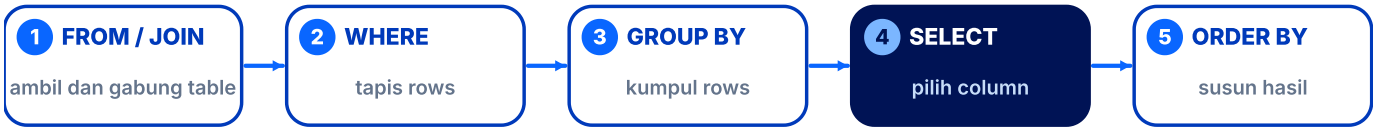
CONTOH GABUNGAN

```
SELECT department_id, AVG(salary) AS average_salary
FROM employees
WHERE salary >= 3500
GROUP BY department_id
ORDER BY average_salary DESC;
```

department_id	average_salary
3	4500
2	4200
1	3650

1. FROM ambil employees
2. WHERE tapis salary >= 3500
3. GROUP BY kumpul ikut department
4. SELECT kira purata
5. ORDER BY susun purata

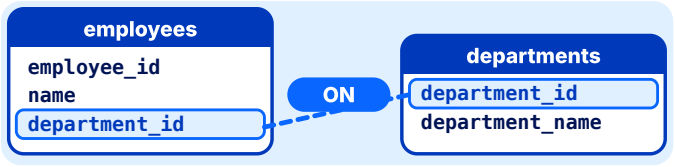
SUSUNAN PEMPROSESAN LOGIK QUERY (DIPERMUDAHKAN)

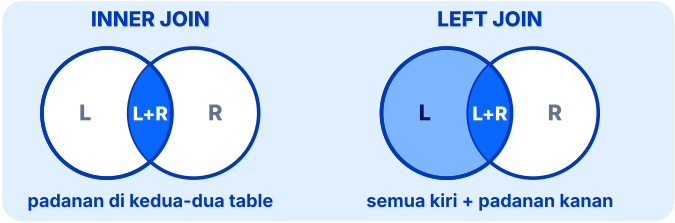


Ni susunan logik yang dipermudahkan, bukan susunan SQL mesti ditulis. Query SQL tetap ditulis bermula dengan SELECT, dan execution sebenar boleh dioptimumkan oleh database engine.

06 JOIN

Gabung rows yang berkaitan daripada dua atau lebih table guna matching condition.





Hubungan: `employees.department_id` sepadan dengan `departments.department_id`.

INNER JOIN

Pulangkan rows yang ada nilai sepadan dalam kedua-dua table.

```
SELECT employees.name, departments.department_name
FROM employees
INNER JOIN departments
ON employees.department_id = departments.department_id;
```

LEFT JOIN

Pulangkan semua rows table kiri dan rows sepadan table kanan. Column kanan yang tak sepadan jadi NULL.

```
SELECT employees.name, departments.department_name
FROM employees
LEFT JOIN departments
ON employees.department_id = departments.department_id;
```

JOIN TYPE	HASIL
INNER JOIN	Rows sepadan daripada kedua-dua table
LEFT JOIN	Semua rows table kiri dan rows sepadan table kanan

Dalam company_db sekarang, kedua-dua JOIN beri 5 rows (setiap pekerja ada department). Bezanya nampak bila ada pekerja tanpa department, lihat bawah.

PENTING

- Klausa ON tentukan cara rows dipadankan.
- JOIN bukan sekadar gabung table yang tak berkaitan.
- Guna explicit JOIN syntax, bukan senarai table dipisahkan koma.

ANDAIAN DI LUAR COMPANY_DB: PEKERJA BARU TANPA DEPARTMENT (DEPARTMENT_ID = NULL). DIPAPARKAN ROWS TERAKHIR SAHAJA.

INNER JOIN: 5 ROWS, NADIA TAK MUNCUL

name	department_name
Aina	Finance
Amir	Marketing

LEFT JOIN: 6 ROWS, NADIA MUNCUL DENGAN NULL

name	department_name
Aina	Finance
Amir	Marketing
Nadia	*NULL*

BILA COMMAND DIGABUNG: JOIN, WHERE DAN ORDER BY

CONTOH GABUNGAN

```
SELECT employees.name,
       departments.department_name,
       employees.salary
FROM employees
INNER JOIN departments
ON employees.department_id = departments.department_id
WHERE employees.salary > 3500
ORDER BY employees.salary DESC;
```

name	department_name	salary
Aina	Finance	4500
Sara	Marketing	4200
Daniel	IT	3800

1 FROM / JOIN gabung table → 2 WHERE tapis salary → 3 SELECT pilih column → 4 ORDER BY susun

Command ni ubah data dalam database. Kena lebih berhati-hati dengan UPDATE dan DELETE.

07 INSERT

Tambah rekod baru ke dalam table.

SYNTAX

```
INSERT INTO table_name (column1, column2)
VALUES (value1, value2);
```

CONTOH

```
INSERT INTO employees
(employee_id, name, department_id, salary, city)
VALUES
(106, 'Nadia', 1, 4000, 'Cyberjaya');
```

Query ni tambah satu rekod pekerja baru.

REKOD BARU

employee_id	name	department_id	salary	city
106	Nadia	1	4000	Cyberjaya

PENTING

- Nyatakan nama column secara explicit.
- Values mesti padan dengan column yang dinyatakan.
- Data type dan constraint mesti dipatuhi.
- Column auto-generated boleh diabaikan jika disokong.

08 UPDATE

Ubah rekod sedia ada.

SYNTAX

```
UPDATE table_name
SET column_name = new_value
WHERE condition;
```

CONTOH

```
UPDATE employees
SET salary = 4000
WHERE employee_id = 101;
```

Ubah salary pekerja dengan ID 101.

AMARAN **Jangan execute UPDATE tanpa semak WHERE.**
Query tanpa WHERE boleh mengubah setiap row dalam table.

BERBAHAYA: TIADA WHERE

```
UPDATE employees
SET salary = 4000;
```

SAFETY CHECKLIST

1. Semak rekod sasaran dengan SELECT dulu.
2. Pastikan condition WHERE betul.
3. Guna transaction bila sesuai.
4. Semak rows terjejas sebelum commit, jika disokong.

09 DELETE

Buang rekod sedia ada daripada table.

SYNTAX

```
DELETE FROM table_name
WHERE condition;
```

CONTOH

```
DELETE FROM employees
WHERE employee_id = 106;
```

Padam rekod pekerja dengan ID 106.

AMARAN **DELETE tanpa WHERE boleh buang setiap row.**
Semua rekod dalam table boleh hilang dengan satu query.

BERBAHAYA: TIADA WHERE

```
DELETE FROM employees;
```

SAFETY CHECKLIST

1. Run SELECT dengan condition yang sama dulu.
2. Sahkan rekod yang betul.
3. Guna transaction bila sesuai.
4. Faham behaviour rollback dan commit database kita.

Q SQL COMMAND QUICK REFERENCE

01 SELECT

```
SELECT column
FROM table;
```

02 FROM

```
FROM table_name;
```

03 WHERE

```
WHERE condition;
```

04 ORDER BY

```
ORDER BY column ASC;
```

05 GROUP BY

```
GROUP BY column;
```

06 JOIN

```
SELECT *
FROM table1
JOIN table2
ON table1.id = table2.id;
```

07 INSERT

```
INSERT INTO table_name (column)
VALUES (value);
```

08 UPDATE

```
UPDATE table_name
SET column = value
WHERE condition;
```

09 DELETE

```
DELETE FROM table_name
WHERE condition;
```

FROM, WHERE, ORDER BY dan GROUP BY ialah klausa yang digabung dengan SELECT, bukan query lengkap sendiri.

+ **INSERT** Tambah records	~ **UPDATE** Ubah records sedia ada	x **DELETE** Buang records

! COMMON SQL MISTAKES

1. Lupa WHERE dalam UPDATE atau DELETE.
2. Guna `= NULL` dan bukan `IS NULL`.
3. Lupa tanda petikan pada string literal.
4. Guna nama column atau table yang salah.
5. Keliru antara GROUP BY dan ORDER BY.
6. Lupa condition ON dalam JOIN.
7. Guna SELECT * tanpa perlu dalam production query yang besar.

SQL SAFETY CHECKLIST: SEBELUM RUN QUERY YANG UBAH DATA

1. Sahkan database yang betul.
2. Sahkan table yang betul.
3. Preview rekod terjejas guna SELECT.
4. Semak condition WHERE.
5. Guna transaction bila sesuai.
6. Semak hasil sebelum commit.

SELECT untuk membaca. INSERT untuk menambah. UPDATE untuk mengubah. DELETE untuk memadam.

Download untuk ulang kaji SQL sebelum lab atau quiz.

Technology, Made Understandable.